Minikube Quickstart for Confluent Platform Core Components

This tutorial provides a step-by-step guide to deploying the core components of the **Confluent Platform** on a local Minikube Kubernetes cluster using **Confluent for Kubernetes (CFK)**.

By the end of this guide, you will have a fully functional Confluent Platform running in **KRaft mode (ZooKeeper-less Kafka)**, enabling you to explore and experiment with:

- **Kafka (KRaft mode)**
- **Schema Registry**
- **Kafka Connect**
- **ksqlDB**
- **REST Proxy**
- **Control Center**

Table of Contents

- 1.0 MacOS prerequisites setup
  - 1.1 Prerequisites
  - 1.2 Install Minikube
  - 1.3 Verify Installation
  - 1.4 Start Minikube
- 2.0 Startup the Confluent Operator (CFK)
  - 2.1 Create the Confluent namespace and set it as the default context for `kubectl`
  - 2.2 Add the Confluent Helm repo and install the Confluent Operator
  - 2.3 Verify the Operator is running
- 3.0 Deploy KRaft broker and controller with CFK
  - 3.1 Set the TUTORIAL_HOME environment variable
  - 3.2 Apply the Confluent Platform manifest
  - 3.3 Verify the platform is running
  - 3.4 Access Control Center
- 4.0 Teardown the CP Core Components
- 5.0 Deploy Provectus' Kafka UI
  - 5.1 Add the Helm repo
  - 5.2 Install Kafka UI
  - 5.3 Verify the installation
  - 5.4 Open the UI
- 6.0 Glossary
- 7.0 Kubernetes Nautical Theme

1.0 MacOS prerequisites setup

1.1 Prerequisites

Need a container driver running. Docker Desktop is a good choice:

```
brew install --cask docker
```

Then launch Docker Desktop and make sure it's running before proceeding.

Install `kubectl`:

```
brew install kubectl
```

1.2 Install Minikube

```
brew install minikube
```

If which minikube fails after install, relink it:

```
brew link --overwrite minikube
```

1.3 Verify Installation

```
minikube version
```

1.4 Start Minikube

CP is very memory and resource intensive, so we need to allocate more resources to the Minikube cluster. The default settings may not be sufficient for running CP components smoothly. We will allocate 6 CPU cores, 20GB of memory, and 50GB of disk space to ensure that the cluster can handle the workload effectively.

```
minikube start --driver=docker --cpus=6 --memory=20480 --disk-size=50g
```

- `--driver=docker`: Use Docker as the container runtime (instead of VirtualBox or Hyper-V).
- `--cpus=6`: Allocate 6 CPU cores to the Minikube cluster.
- `--memory=20480`: Allocate 20GB of memory to the Minikube cluster.
- `--disk-size=50g`: Allocate 50GB of disk space to the Minikube cluster.

Then verify that Minikube is running:

```
minikube status
kubectl get nodes
```

2.0 Startup the Confluent Operator (CFK)

2.1 Create the Confluent namespace and set it as the default context for `kubectl`

```
kubectl create namespace confluent
kubectl config set-context --current --namespace=confluent
```

2.2 Add the Confluent Helm repo and install the Confluent Operator

```
helm repo add confluentinc https://packages.confluent.io/helm
helm repo update
helm upgrade --install confluent-operator confluentinc/confluent-for-
kubernetes --namespace confluent
```

2.3 Verify the Operator is running

```
kubectl get pods -n confluent
```

If all goes well you should see the following output:

```
NAME                                  READY   STATUS    RESTARTS   AGE
confluent-operator-66b887b979-frj42   1/1     Running   0          15s
```

The CFK operater is now running and ready to manage Confluent Platform resources in the cluster.

3.0 Deploy KRaft broker and controller with CFK

3.1 Set the TUTORIAL_HOME environment variable

```
export
TUTORIAL_HOME="https://raw.githubusercontent.com/confluentinc/confluent-
kubernetes-examples/master/quickstart-deploy/kraft-quickstart"
```

The `TUTORIAL_HOME` environment variable is being set to a raw GitHub URL that points to the directory containing the Kubernetes manifest for deploying Confluent Platform in KRaft mode. This allows us to reference the manifest file directly from that URL in the next step when we apply it with `kubectl`.

Sets an environment variable pointing to a raw GitHub URL for Confluent's official Kubernetes examples repo — specifically the KRaft quickstart directory. KRaft is Kafka's ZooKeeper-free mode (Kafka Raft metadata).

> **To browse the manifest file directly, you can visit:** https://github.com/confluentinc/confluent-kubernetes-examples/tree/master/quickstart-deploy/kraft-quickstart

3.2 Apply the Confluent Platform manifest

```
kubectl apply -f $TUTORIAL_HOME/confluent-platform-c3++.yaml
```

> The `raw.githubusercontent.com` URL works when you append a specific filename to it, allowing you to directly access the raw content of that file. When you use `kubectl apply -f` with a URL, `kubectl` can fetch the YAML manifest directly from that URL and apply it to your Kubernetes cluster without needing to download the file manually first. This is a convenient way to deploy Kubernetes resources directly from a GitHub repository.

Applies a Kubernetes manifest file directly from that GitHub URL, `kubectl` can fetch and apply remote YAML files without downloading them first. The c3++ in the filename likely refers to Control Center (C3) with additional/enhanced configuration (the ++ suggesting an extended or "plus" variant).

> This is spinning up the Confluent Platform on Kubernetes using the Confluent Operator (CFK - Confluent for Kubernetes) in KRaft mode. It would typically deploy:
>
> - Kraft Controllers
> - Kafka Brokers
> - Schema Registry
> - Kafka Connect
> - ksqlDB
> - REST Proxy
> - Control Center (C3) — the Confluent monitoring UI

3.3 Verify the platform is running

Watch the pods in the confluent namespace until all components are up and running (this will take a few minutes):

```
kubectl get pods -n confluent -w
```

You're looking for all pods to be in the "Running" state.

Final status checker for Kubernetes Operators:

```
kubectl get pods -n confluent
```

You should see all pods at `1/1` or `3/3 Running`. Then open the Control Center UI to verify the platform is healthy.

Open Control Center:

```
kubectl port-forward -n confluent controlcenter-0 9021:9021
```

3.4 Access Control Center

Go to the Control Center UI dashboard:

```
http://localhost:9021/
```

4.0 Teardown the CP Core Components

To clean up the resources created by this tutorial, you can delete the Confluent Platform deployment and the Confluent Operator:

```
kubectl delete -f $TUTORIAL_HOME/confluent-platform-c3++.yaml
helm uninstall confluent-operator -n confluent
kubectl delete namespace confluent
```

5.0 Deploy Provectus' Kafka UI

5.1 Add the Helm repo

```
helm repo add kafka-ui https://provectus.github.io/kafka-ui-charts
helm repo update
```

5.2 Install Kafka UI

```
helm upgrade --install kafka-ui kafka-ui/kafka-ui \
  --namespace confluent \
  --set yamlApplicationConfig.kafka.clusters[0].name="confluent" \
  --set
yamlApplicationConfig.kafka.clusters[0].bootstrapServers="kafka:9092" \
  --set
yamlApplicationConfig.kafka.clusters[0].schemaRegistry="http://schemaregis
try:8081" \
  --set
yamlApplicationConfig.kafka.clusters[0].kafkaConnect[0].name="connect" \
  --set
yamlApplicationConfig.kafka.clusters[0].kafkaConnect[0].address="http://co
```

```
nnect:8083" \
  --set yamlApplicationConfig.auth.type="DISABLED" \
  --set yamlApplicationConfig.management.health.ldap.enabled="false"
```

The service names (`kafka`, `schemaregistry`, `connect`) are the internal Kubernetes DNS names assigned by CFK, they only resolve from within the cluster, which is why Kafka UI runs inside the same namespace.

5.3 Verify the installation

```
kubectl get pods -n confluent | grep kafka-ui
```

Wait until the pod shows `Running` before port-forwarding.

5.4 Open the UI

```
kubectl port-forward -n confluent svc/kafka-ui 8080:80
```

Then open `http://localhost:8080` in your browser. Note the asymmetry: the service listens on port `80` internally, forwarded to `8080` locally.

6.0 Glossary

Term	Description
CFK	Confluent for Kubernetes, a set of Kubernetes Operators for deploying and managing Confluent Platform on Kubernetes.
CRD	Custom Resource Definition, a way to extend Kubernetes capabilities by defining custom resources.
Operator	A method of packaging, deploying, and managing a Kubernetes application.
Controller	A control loop that watches the state of your cluster and makes or requests changes as needed.
Reconciliation	The process of ensuring that the current state of the cluster matches the desired state defined by a resource.
Finalizer	A mechanism to perform cleanup before a resource is deleted.
Webhook	A way to extend Kubernetes API by intercepting API requests and modifying them or validating them.
CR	Custom Resource, an instance of a CRD that represents a specific configuration or state in the cluster.

7.0 Kubernetes Nautical Theme

The nautical theme runs throughout the Kubernetes ecosystem:

Term	Description
Kubernetes	Helmsman/pilot
Helm	The ship's wheel
Charts	Nautical maps (Helm packages are called "charts")
Harbor	A container registry (like a port where ships dock)
Fleet	A multi-cluster management tool